

# Project : Custom VPC Setup with Bastion Host and NAT Gateway

## Services Used:

- Amazon VPC
- Amazon EC2
- Internet Gateway
- NAT Gateway
- Security Groups
- Route Tables

## Description :

This project shows how to create a secure network in AWS using a custom VPC. In this setup, we create two types of subnets:

- Public subnet (for internet access)
- Private subnet (for secure internal use)

A Bastion Host is used as a middle server to safely connect to private instances using SSH.

A NAT Gateway is used to give internet access to private instances, but without making them public or exposing them directly to the internet.

🔗 In simple terms:

- Bastion Host = secure entry point
- NAT Gateway = safe internet access for private servers

## Goal (Simple):

- Create a secure network
- Allow access to private server safely
- Give internet to private server without public access

## Skills Learned:

- How to create VPC
- How to create subnets
- How routing works
- How to use Bastion Host
- How NAT Gateway works
- How to connect using SSH

## Definition:

A Custom VPC with Bastion Host and NAT Gateway is a secure cloud network setup where:

- Public subnet hosts internet-facing resources

- Private subnet hosts internal resources
- Bastion Host enables secure SSH access
- NAT Gateway allows outbound internet access for private instances

## Scope of the Project:

This project includes:

- Creation of a custom VPC with CIDR block
- Setup of public and private subnets
- Configuration of Internet Gateway for public access
- Deployment of Bastion Host in public subnet
- Deployment of private EC2 instance in private subnet
- Configuration of NAT Gateway for secure internet access
- Implementation of routing tables
- Secure access using SSH **Methodology:**

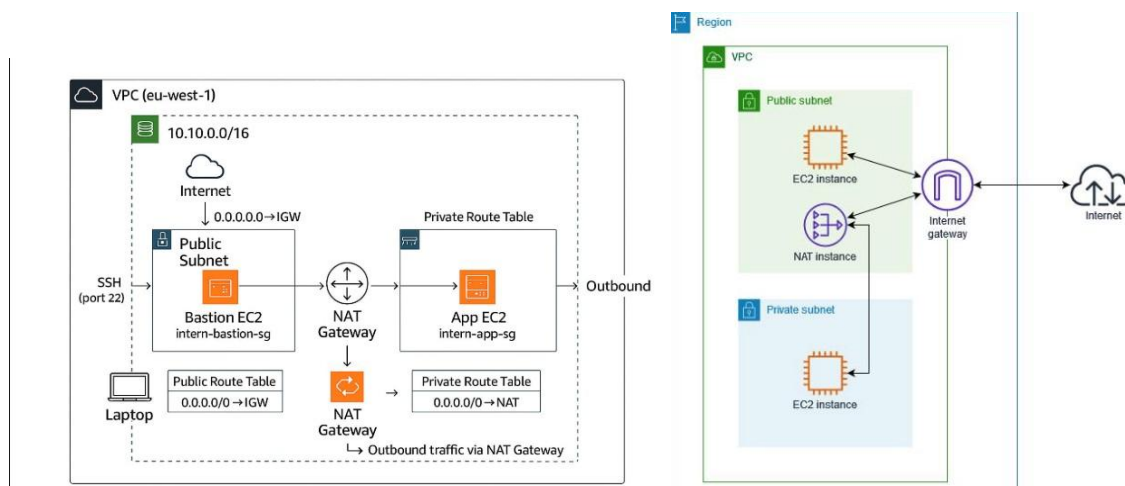
### 1. Bastion Host + NAT Gateway (Best Method)

- Secure architecture using public and private subnets
- Bastion Host used as jump server
- NAT Gateway used for internet access for private instances
- Highly secure and industry standard
- No direct exposure of private instances

### Direct Public Access (Not Recommended)

- Instances are given public IP
- Direct SSH access from internet
- Less secure and not suitable for production

Architecture:



## Implementation Steps to Do the Project:

### Step1: Create VPC

VPC is your private network in AWS where all your resources will run

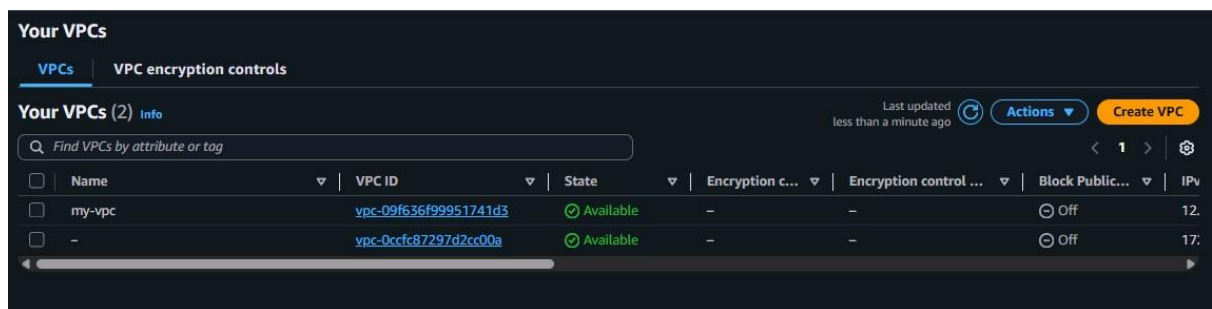
Go to AWS Console - Search and open VPC - Click Create VPC

Fill Details:

- Name: **custom-vpc**
- IPv4 CIDR block: **10.0.0.0/16**

10.0.0.0/16 is called a CIDR block

- /16 means how many bits are used for network part
- It defines size of your network (how many IPs you get)



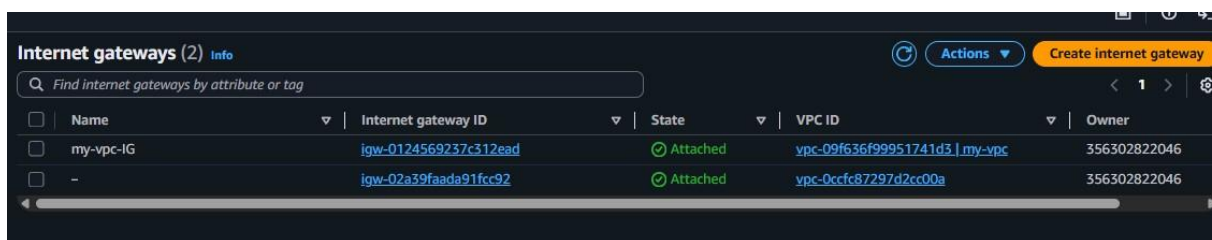
**Step 2: Create Internet Gateway:** Internet Gateway (IGW) is used to connect your VPC to the internet—Without IGW → No internet

#### Steps to Create IGW:

1. Go to **VPC Dashboard**
2. Click **Internet Gateways**
3. Click **Create Internet Gateway**
4. **Give Name:**
  - **My-igw**
  - Click **Create**

Click **Actions** → **Attach to VPC**

Choose your VPC (**custom-vpc**)



**Step 3: Create Subnets:** Subnet is a small part of your VPC network (VPC = big network) Subnet = small section inside.

#### Steps to Create Subnets:

Create 2 Subnets

## 1. Public Subnet

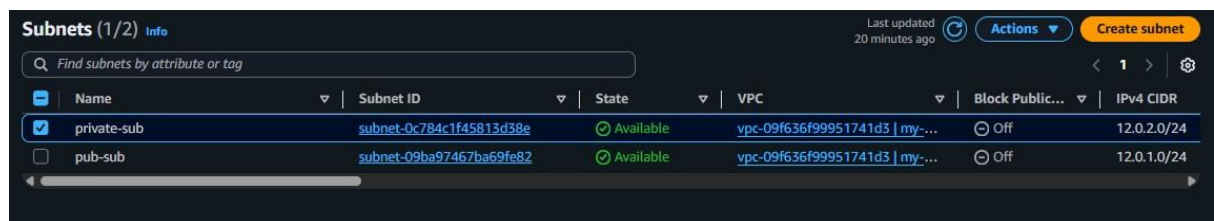
- VPC: **my-vpc**
- Name: **pub-sub**
- Availability zone: **south1**
- CIDR: **10.0.1.0/24**
- Enable: **Auto-assign Public IP = ON**

This subnet will have internet access

## 2. Private Subnet

- VPC: **my-vpc**
- Name: **private-sub**
- Availability zone: **south1**
- CIDR: **10.0.2.0/24**

This subnet is secure (no direct internet)



The screenshot shows the AWS Subnets console. At the top, it says 'Subnets (1/2)' and 'Info'. There's a search bar and a 'Find subnets by attribute or tag' button. On the right, it says 'Last updated 20 minutes ago', 'Actions', and 'Create subnet'. Below this is a table with columns: Name, Subnet ID, State, VPC, Block Public..., and IPv4 CIDR. There are two subnets listed: 'private-sub' and 'pub-sub'. Both are in 'Available' state and are associated with 'vpc-09f636f99951741d3 | my-...'. The 'private-sub' has a CIDR of '12.0.2.0/24' and 'Block Public...' is 'Off'. The 'pub-sub' has a CIDR of '12.0.1.0/24' and 'Block Public...' is 'Off'.

| Name        | Subnet ID                | State     | VPC                            | Block Public... | IPv4 CIDR   |
|-------------|--------------------------|-----------|--------------------------------|-----------------|-------------|
| private-sub | subnet-0c784c1f45813d38e | Available | vpc-09f636f99951741d3   my-... | Off             | 12.0.2.0/24 |
| pub-sub     | subnet-09ba97467ba69fe82 | Available | vpc-09f636f99951741d3   my-... | Off             | 12.0.1.0/24 |

## Step 4: Route Tables: Route Table decides where your traffic goes

- It controls internet access
- It connects subnets to IGW or NAT

## Create 2 Route Tables: 1-

### Public Route Table:-

#### Create:

- Name: **public-rt**
- VPC: **my-vpc**

#### ❖ Add Route:

Go to **Routes** → **Edit routes** → **Add**  
0.0.0.0/0 → Internet Gateway (IGW)

- ❖ This allows **internet access**

## Associate Subnet:

- Go to Subnet Associations
- Select public-subnet

**Edit routes**

| Destination | Target           | Status | Propagated | Route Origin     |
|-------------|------------------|--------|------------|------------------|
| 12.0.0.0/16 | local            | Active | No         | CreateRouteTable |
| 0.0.0.0/0   | Internet Gateway | Active | No         | CreateRoute      |

Buttons: Add route, Cancel, Preview, Save changes

**Edit subnet associations**

Change which subnets are associated with this route table.

| Name        | Subnet ID                | IPv4 CIDR   | IPv6 CIDR | Route table ID                     |
|-------------|--------------------------|-------------|-----------|------------------------------------|
| private-sub | subnet-0c784c1f45813d38e | 12.0.2.0/24 | -         | rtb-00dd2f2eb0633cd86 / Private-RT |
| pub-sub     | subnet-09ba97467ba69fe82 | 12.0.1.0/24 | -         | rtb-0e4f3016ea47bb538 / Public-RT  |

Selected subnets: subnet-09ba97467ba69fe82 / pub-sub

Buttons: Cancel, Save associations

## 2. Private Route Table

- **Create:**
- Name: **private-rt**
- VPC: **my-vpc**

## Associate Subnet:

- **Select private-subnet**
- **Important:-DO NOT add IGW here Private subnet should NOT have direct internet**

**Route tables (2/4)**

| Name       | Route table ID        | Explicit subnet associ... | Edge associations | Main | VPC                        |
|------------|-----------------------|---------------------------|-------------------|------|----------------------------|
| Public-RT  | rtb-0e4f3016ea47bb538 | subnet-09ba97467ba69f...  | -                 | No   | vpc-09f636f99951741d3   my |
| Private-RT | rtb-00dd2f2eb0633cd86 | subnet-0c784c1f45813d...  | -                 | No   | vpc-09f636f99951741d3   my |
| -          | rtb-048a2d59931846a4d | -                         | -                 | Yes  | vpc-09f636f99951741d3   my |
| -          | rtb-07e1d69cc8fe48b94 | -                         | -                 | Yes  | vpc-0ccfc87297d2cc00a      |

Buttons: Actions, Create route table

**Step 5: Create NAT Gateway:-** NAT Gateway gives internet access to private instances, But keeps them secure (no public IP)

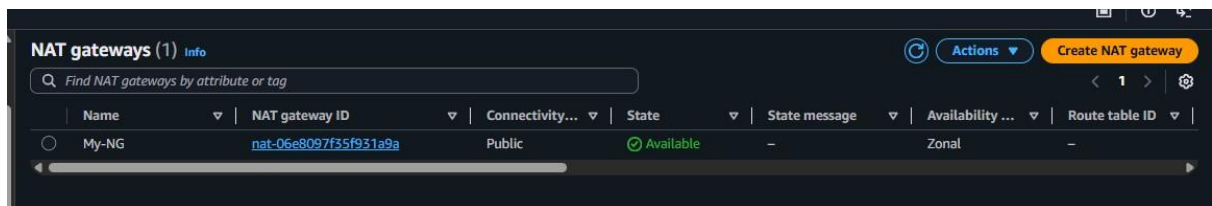
## Steps to Create NAT Gateway

1. Go to **VPC Dashboard**
2. Click **NAT Gateways**
3. Click **Create NAT Gateway**

## Fill Details:

- **Name:** **my-NG**
- **Availability mode:** **Zonal**
- **Subnet:** **public-subnet .. (VERY IMPORTANT)**
- **Elastic IP:** Click **Allocate Elastic IP**

Then click **Create NAT Gateway**

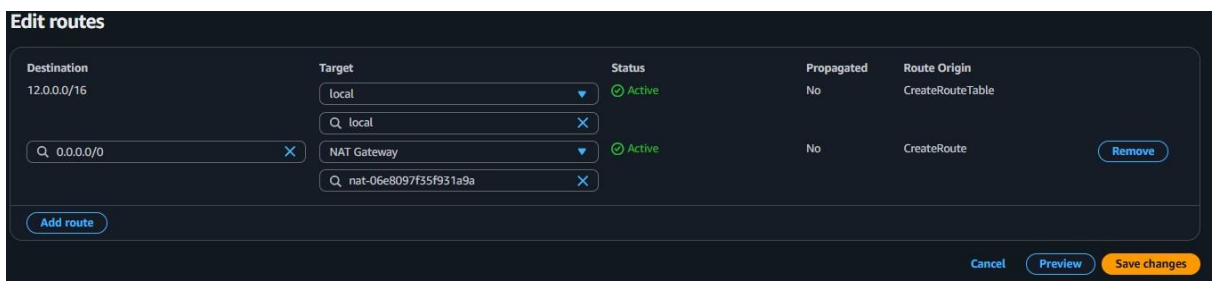


## Update Private Route Table (MOST IMPORTANT STEP)

Go to Private Route Table → Routes → Edit

Add: 0.0.0.0/0 → NAT Gateway

Now private instances get the internet.



## Step 7: Launch Bastion Host (Public & private instance)

### What is Bastion?

Bastion Host = Normal EC2 instance in public subnet

It is just:

- A server with public IP
- Used to connect to private server...  
You cannot connect directly to private server, so you go like this:  
Your Laptop → Bastion → Private Server

### Where is Bastion in YOUR Project?

The EC2 instance you created with:

- Public Subnet
- Public IP
- SSH allowed

THAT is your Bastion Host

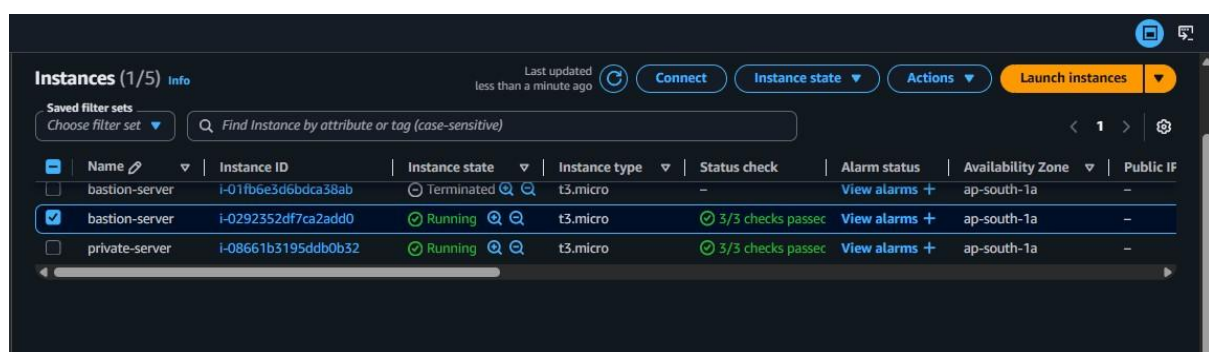
## What is SSH?

SSH = **Secure Shell**

It is a way to connect and control a server from your laptop

Example `ssh -i key.pem ec2-user@<public-ip>`

## Steps to create bastion public instance:



- Go to EC2 → Launch Instance
- Name: **bastion-server**
- AMI: Amazon Linux
- Instance type: t2.micro
- Create key: bastionkey
- Same key for both bastion server and private

1. **Private Key (.pem file) → stays with YOU (your laptop)**
2. **Public Key → stored inside EC2 instance**

## Network Settings:

- VPC → my-vpc
- Subnet → **Public Subnet**
- Auto-assign Public IP → **Enable**

## Security Group: default Security group (bastion-SG)

Add rule:

- Type: SSH
- Port: 22
- Source: (custom: 0.0.0.0/0)
- Security group → keep default and change name.

## Steps to create Private instance :

- Go to EC2 → Launch Instance
- Name: **private-server**
- AMI: Amazon Linux

- Instance type: t2.micro
- Create key: bastionkey
- Same key for both bastion server and private

#### Network Settings:

- VPC → my-vpc
- Subnet → **Private Subnet**
- Public IP → **Disable**

#### Security Group (VERY IMPORTANT):

keep default Security group (private-instance-SG)

Add:

- Type: SSH
- Source type: (custom )
- **Source : Bastion Security Group** (NOT 0.0.0.0/0)

### Step 8: Connect to Bastion server → Then Private

- Open the **Git-bash terminal**:
- # ls → for list you key should be there (bastionkey.pem) Give read permission of key # chmod 400 "bastionkey.pem"
- #ssh -i "bastionkey.pem" ec2-user@13.201.135.176

```
Mudasir Ibn Usuf@bhat-mudasir MINGW64 ~
$ chmod 400 bastionkey.pem

Mudasir Ibn Usuf@bhat-mudasir MINGW64 ~
$ ssh -i "bastionkey.pem" ec2-user@13.201.135.176
The authenticity of host '13.201.135.176 (13.201.135.176)' can't be established.
ED25519 key fingerprint is: SHA256:cfq67Pltjbx7PjornynzvLlo60CNRZHJGGHhBfzbzcs
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.201.135.176' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

#_
~\  #####          Amazon Linux 2023
~\  #####\
~\  #####|
~\  \#/_
~\  V~'  '→ https://aws.amazon.com/linux/amazon-linux-2023
~\  _-+
~\  _/
~\  /m/'

Last login: Fri Mar 27 16:39:16 2026 from 13.233.177.4
[ec2-user@ip-12-0-1-140 ~]$ ls
```

successfully logged into your EC2 Bastion public Host [ec2-user@ip-12-0-1-140 ~]\$

### Next step (VERY IMPORTANT in your setup):

Now you should connect from bastion → private EC2 instance

Step 1: Find private EC2 IP



Go to *AWS Console*:

- EC2 → Private instance
- Copy Private IPv4 address Example: 10.0.2.15

## Step 2: You need private key on bastion

Right now, our key is on our **Windows machine**, not inside the bastion.

We must do ONE thing: Go inside the terminal and create new windows on git terminal.

- `#ls you will see bastionkey.pem` • `#cat bastionkey.pem`
- Copy the content of this key.
- Go into the main terminal where the bastion is login.
- `#touch new-private-key`
- `#Vi new-private-key` past the content in it → `:wq`
- `#chmod 400 new-private-key` • Go to private instance copy SSH :
- `ssh -i "new-private-key" ec2-user@12.0.2.24`

```
Mudasir Ibn Usuf@bhat-mudasir MINGW64 ~  
$ ssh -i "bastionkey.pem" ec2-user@13.201.135.176  
The authenticity of host '13.201.135.176 (13.201.135.176)' can't be established.  
ED25519 key fingerprint is: SHA256:cfaq67Pltjbx7PjornynzvLlo6OCNRZHJGGHbfzbcsc  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '13.201.135.176' (ED25519) to the list of known hosts.  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html
```

```
_#_
~\   ####_      Amazon Linux 2023
    \#####\
     \|###|
      \|#/ |_____|
       V~'|___>
         ._. /
        _/_/\_/
          m/'

Last login: Fri Mar 27 16:39:16 2026 from 13.233.177.4
[ec2-user@ip-12-0-1-140 ~]$ ls
[ec2-user@ip-12-0-1-140 ~]$ ls
[ec2-user@ip-12-0-1-140 ~]$ touch new-key
[ec2-user@ip-12-0-1-140 ~]$ vi new-key
[ec2-user@ip-12-0-1-140 ~]$ ls
new-key
[ec2-user@ip-12-0-1-140 ~]$ chmod 400 new-key
[ec2-user@ip-12-0-1-140 ~]$ ssh -i "new-key" ec2-user@12.0.2.24
The authenticity of host '12.0.2.24 (12.0.2.24)' can't be established.
ED25519 key fingerprint is: SHA256:vbeTcxDu/76vEp1bh19QAEkoAbmnXOashrQLUjGrGfg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '12.0.2.24' (ED25519) to the list of known hosts.
```

```
_#_
~\   ####_      Amazon Linux 2023
    \#####\
     \|###|
      \|#/ |_____|
       V~'|___>
         ._. /
        _/_/\_/
          m/'

[ec2-user@ip-12-0-2-24 ~]$ Read from remote host 13.201.135.176: Connection reset by peer
Connection to 13.201.135.176 closed.
client_loop: send disconnect: Connection reset by peer
```

```
Mudasir Ibn Usuf@bhat-mudasir MINGW64 ~  
$
```

## Troubleshooting:

Issue 1: Unable to install packages in Private Instance (yum/apt not working)

Solution:

Private instance had no internet access

Created a NAT Gateway in public subnet

Attached an Elastic IP to NAT Gateway

Updated private route table:

0.0.0.0/0 → NAT Gateway

After this, package installation worked successfully

Issue 2: SSH connection failed due to key permission error

Solution:

Initially forgot to set correct permissions for .pem file

Fixed by running:

```
chmod 400 bastionkey.pem
```

After updating permissions, SSH connection worked properly

Issue 3: Unable to SSH into Bastion Host

Solution:

Checked Security Group allows SSH (Port 22)

Verified correct public IP

Used correct key and username (ec2-user)

## Conclusion:

This project demonstrates a secure AWS network setup using a custom VPC with public and private subnets. It uses a Bastion Host for secure access and a NAT Gateway for safe internet connectivity. It also helped in understanding real-world troubleshooting like SSH permission issues and private instance internet access.

